

Report

Railway Scheduling - Transfer Coordination

Kwun Pok Wong
Masters in Cognitive Systems
University of Potsdam

Tuan Vinh Nguyen
Bachelors in Computational Science
University of Potsdam

September 25, 2025

1 Introduction

Growing research on transfer coordination problem (TCP) of public transport has been shown in recent years due to its importance in improving the logistics of large-scale transportation systems. The ease of transfer highly influences the travellers' level of satisfaction with their choice of transport [1]. Therefore, measures can be implemented by transport agents such that the subjective experience of passengers can be improved, such as maintaining a high level of station services or providing sufficient seating at stations. Additionally, the reduction of overall travel time by adjusting multiple variables of the transport operations is a systematic approach to enhance passengers' experience. TCP is a multi-variable problem, that is, multiple factors affect the optimization of the objective in a mathematical model. For example, the waiting cost, the longest and shortest waiting times, and the period of operation are all important variables that determine the optimization outcome. Studies have adopted different strategies to reduce transfer waiting time at a tactical level [2]. In this paper, we extend on previously proposed solutions that utilize mathematical programming by using an Answer Set Programming (ASP) implementation on a Flatland environment, which was originally created for multi-agent reinforcement learning problems by scheduling railways [3]. An urban rail transit (URT) system is simulated with path-finding, where trains are required to reach assigned stations and eventually reach the end terminal.

1.1 Synchronization Approach to TCP

In this project, we focus on maximizing the number of synchronized connections at transfer stations as opposed to minimizing the transfer time directly. Some studies adopt an analytical approach by expressing the problem with continuous functions, but we attend to integer programming that produces realistic solutions [4]. Additionally, the maximization of synchronized connections is considered as the objective in this report to enable permissibility and flexibility of defining the transfer waiting time [5]. Here, we define a synchronized connection as an instance for which the difference in time between the departure vehicle and the arrival vehicle falls within a specific range of waiting time. This avoids expressing the resulting waiting time as a single constant, which is highly unrealistic. Overall, modeling the problem with integer constraints and defining the synchronization objective allows a natural formulation and solution to the problem with ASP encoding, as seen in Section 3.

1.2 Extension to Flatland

The original Flatland problem consists of a 2D grid with limited transitions between neighboring cells making up the railway network, as well as multiple trains or “agents” who must travel in this network whilst maximizing a common global reward [6]. This environment was extended to include train lines which have trains running the same route, with the global goal being set to maximizing the number of synchronized connections.

The representation of TCP in Flatland is decided with several considerations. Firstly, its simplicity is maintained so that the problem is solved efficiently with minimal grounding and solving time in ASP. Secondly, the representation is sufficient to represent TCP itself and additional problems depending on the specific scenario. Thirdly, it mimics reality such that it is not too simplified or distorted for the sake of achieving a desirable solution.

1.3 Contributions of the Report

This report contributes to solving TCP on several fronts:

1. Representations of TCP in a modified Flatland environment are proposed.
2. ASP encoding for TCP of various scenarios in Flatland is explored. ASP allows a simplistic expression of the problem with declarative constraints, and a simultaneous solution to two common problems in railway scheduling. ASP also potentially provides a globally optimized solution, as opposed to local search strategies such as the genetic algorithm (GA).
3. Visualization of solution sets is also provided as a development tool.

We find that ASP is capable of solving the transfer coordination problem together with path-finding efficiently with self-defined examples, but future work is required to determine its scalability.

2 Problem Formulation

Our problem formulation of TCP follows [5] as an example of the synchronization approach. We assume that the transfer times of passengers are varied, but the range is known. The capacity of each train is assumed to be unlimited. We are interested in the case where headway is constant for each line, meaning that the departure time between any two consecutive trains of each line is the same.

2.1 TCP Objective

Let $i = 1, 2, \dots, l \in I$ denote the lines of the URT, and $j \in J(i)$ as the set of lines that possibly connect to line i . Let $n = 1, 2, \dots, m \in N$ be the synchronized nodes, which are the stations that allow synchronized connections. T is the planning horizon that defines the period in which the trains can depart from the starting terminal of each line. f^i and f^j are the total number of trains for line i and j , respectively. The headway h^i is the constant time separation between trains of line i . The dwell time at a synchronized node of line i is denoted by d^i . Let Y_{pqn}^{ij} be a binary variable representing a synchronized connection where

$$Y^{ij} = \begin{cases} 1 & \text{if the train } q \text{ of line } j \text{ can connect the} \\ & \text{train } p \text{ of line } i \text{ at station } n \\ 0 & \text{otherwise} \end{cases}$$

Then the objective of synchronized transfers is

$$F_{SCSO}(Y) = \max \sum_{i \in I} \sum_{j \in J(i)} \sum_{n \in N^{ij}} \sum_{p=1}^{f^i} \sum_{q=1}^{f^j} Y_{pqn}^{ij} \quad (1)$$

which translates to the maximization of the number of synchronized connections with a given period T and constant headways $\{h^i | i \in I\}$.

2.2 Constraints for TCP

As defined previously, each synchronized connection is constrained by the range of waiting time. Specifically, any combination of (i, j, p, q) must follow

$$D_{q,n}^j - (A_{p,n}^i + E_n^{ij}) \geq w_n - M \cdot (1 - Y_{pqn}^{ij}) \quad (2)$$

$$D_{q,n}^j - (A_{p,n}^i + E_n^{ij}) \geq w_n - M \cdot (1 - Y_{pqn}^{ij}) \quad (3)$$

$D_{q,n}^j$ is the departure time of train q of line j at synchronization node n , and $A_{p,n}^i$ is the arrival time of train p of line i at synchronization node n . e and E are the shortest and the longest walking transfer times, respectively. w and W are the shortest and the longest waiting times, respectively. M is an arbitrarily large number. Equation (2) states that the difference between the arrival at the connecting platform by the passenger with the longest transfer time and the departure time of the connecting train must not be less than the shortest waiting time. Equation (3) holds a similar statement. Unlike the original definition in [5], the transfer times and the waiting times are invariant to the connection lines and the synchronization nodes for simplicity.

Since we assume that the headway is constant, the only decision variables are the departure time of the first train of each line. The departure times of subsequent trains are shifted accordingly.

2.3 Additional Constraints

Three scenarios are considered when tackling TCP in Flatland. We first consider the situation in which no path-finding between two terminals is required for any line. In other words, each line follows a predetermined route given by an instance, as detailed below. It is noted that collision is neglected in this case, hence entailing necessary modification to the original Flatland environment.

In the second situation, we combine both TCP and path-finding by omitting predetermined routes and including stations that are not synchronized nodes assigned to each line. The route for each line is then decided by the ASP solver, together with the encoding, such that all non-synchronization and synchronization stations for that line are passed through. Trains on each line should still reach the starting and ending terminals. This combined problem is formulated in this way, such that the capability of ASP in solving TCP and path-finding can be evaluated. Collision is again disabled in this case.

The last situation is almost identical to the first case, with two exceptions being that collision is enabled and each line is assumed to have bidirectional trains. Collision is defined more loosely than that in the original Flatland problem in the sense that only trains travelling in the same direction and landing on the same cell at the same time are considered to be colliding with each other. This is equivalent to assuming that each cell has at most one separate track for trains that travel in each direction. This definition of collision allows trains belonging to the same line but travelling in opposite directions or trains that travel perpendicularly to each other at a junction to

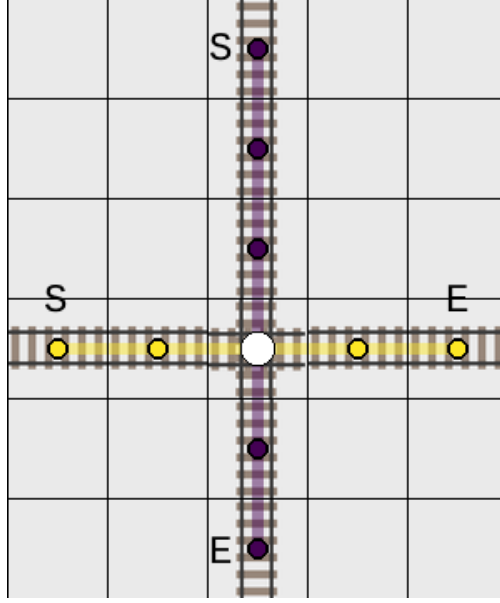


Figure 1: Simple instance for TCP

Flatland Equivalent	TCP
train(ID)	line(LID), train(LID, TID)
start(ID, (X,Y), Dep, Dir)	start(LID, (X,Y), Dir)
end(ID, (X,Y), Arr)	end(LID, (X,Y))

Table 1: Equivalent Atoms - Flatland & TCP

be represented in a modified Flatland instance, while forcing trains travelling in the same direction to possibly share a track.

For all scenarios, each train of a line is assumed to follow the same route, and the routes do not form loops in the graph representation of the environment. Also, trains do not stop unless they reach a station.

2.4 Instance Representation in ASP

Overall, our instances don't differ significantly from those of Flatland, as seen in Figure 1. Two perpendicular train lines are featured and color-coded, with white circles representing the synchronized nodes. Starting and ending cells of each line are marked with the letter S and E, respectively. The exact ASP code for this instance can be found in Listing 1 in Appendix A.

Briefly, cell(X,Y) atoms, as required by the original Flatland problem, define the positions and the types of the tracks. Directions and restrictions for train transitions as well as actions are also preserved. Additionally, we have made light modifications to certain atoms to support train lines, as seen in Table 1.

New atoms such as headway, dwell, sync are introduced in order to solve TCP. The meaning of additional atoms is described in the following:

- headway(LID,T): Time interval T between trains on the same line LID
- dwell(LID,T): A train on line LID would dwell for T time on synchronized nodes

- `sync(NID, (X,Y))`: Synchronized node NID is on cell (X,Y)
- `station(LID, (X,Y))`: Additional stations that trains on line LID must pass through
- Additionally, `route/3` atoms, if they exist, describe predetermined routes that a specific line follows.
- `period(T)`: Total time period considered
- Constants `swait`, `lwait`: shortest and longest waiting period for transfer
- Constants `stransfer`, `ltransfer`: shortest and longest transfer time

3 ASP Encoding

Conforming to the specifications of Flatland, we provide atoms that represent the actions of each train at a specific time.

```

1 1 >= {action(train(L, 1), A, T) : command(A)} :- at(train(L, 1), T, (X, Y), _).
2 action(train(L, N), A, T + TT) :- N != 1, shift(L, N, TT), action(train(L, 1), A, T),
   ↪ headway(L, H).

```

Line 1 is a generator that chooses an action (forward, turn_left, turn_right, wait) for the first train of each line. Trains do not have any actions once they reach the end terminal. Since every train on the same line follows the same route, it can be deduced that the action atoms for subsequent trains are the same, except that they are separated by multiples of headway. Line 2 deterministically assigns actions to subsequent trains by shifting the time by $TT = H * (N - 1)$, where H is the headway of the line, and N is the number of the train.

These actions, however, are not chosen without restrictions. They follow the specifications of Flatland, meaning that the choice is subject to the direction of the train, the position of the train, and the track type.

```

3 coord_dir(ID, w, front, (e,w)) :- track_dir(ID, (e,w)).
4 coord_dir(ID, s, left, (n,e)) :- track_dir(ID, (n,e)).
5 coord_dir(ID, s, right, (n,w)) :- track_dir(ID, (n,w)).

```

In the above sampled code, we define the absolute direction (n, e, s, w) and the relative direction (front, left, right) of the train after an action, based on the track type. Line 3, for example, states that if the track type permits passage from east to west, then the resulting absolute direction is westward, and the relative direction is ‘front’.

Then, we translate these permissible directions, which depend on the track type and the train direction, into possible actions based on constraints such as below:

```

6 possible_action(ID, D, move_left, (D2, D3)) :- coord_dir(ID, D, left, (D2, D3)),
   ↪ coord_dir(ID, D, front, _).

```

Line 6 states that for a given train direction and a track type, if the resulting direction can be ‘left’ or ‘front’, then the action ‘move_left’ is possible. Finally, these possible_action atoms restrict the choices of actions:

```

7 :- action(train(L, 1), A, T), at(train(L, 1), T, (X, Y), D), not possible_action(TID, D,
  ↪ A, _), cell((X, Y), TID).

```

where an action is not allowed to be chosen if it is not possible with a given train direction and track type.

This way of generating actions increases the scalability of the code since it can be generalized to new track types. When compared to hardcoding actions for every track type, it results in an increased grounding time. However, it is not significant enough to disrupt our current objectives, so this implementation is retained.

The positions and directions of the trains are propagated as below:

```

8 at(train(L, 1), T + 1, (X, Y - 1), w) :- action(train(L, 1), A, T), at(train(L, 1), T,
  ↪ (X, Y), D), cell((X, Y), TID), possible_action(TID, D, A, (_, w)), A != wait,
  ↪ cell((X, Y-1), _).

```

where the subsequent position and direction are determined by the action, the initial position and the direction of the train, and the track type.

3.1 TCP without Path-Finding

We start by picking a departure time of the first train for each line L within the range of the headway:

```

9 1 = {first_start(L, 1, T) : T = 0..GT, headway(L, HT), T <= HT} :- period(GT), line(L).

```

It is reminded that these are the only decision variables in the problem.

Then, the auxiliary atoms shift/3, are introduced to indicate the shift in time of the subsequent trains:

```

10 shift(L, N, TT) :- line(L), time(N), time(TT), first_start(L, 1, T), headway(L, H),
  ↪ period(GT), TT = (N - 1) * H, N > 1, TT + T <= GT.

```

With the starting times of the first trains and the shift/3 atoms, we can now define positional and temporal atoms, including 1) the initial positions and directions of the train, 2) the arrival time to each station, and 3) the departure time from each station:

```

11 % define auxiliary atoms
12 % initialize position of the first train
13 at(train(L, 1), T, (X, Y), D) :- first_start(L, 1, T), start(L, (X, Y), D).
14 % deterministically assign positions to the rest of the trains
15 at(train(L, N), T + TT, (X, Y), D) :- at(train(L, 1), T, (X, Y), D), shift(L, N, TT), N
  ↪ != 1.
16 % define arrival time as the earliest timestep at which the train stays at the
  ↪ synchronized node
17 arrive(train(L, 1), SN, ET) :- sync(SN, (X, Y)), ET = #min {T: at(train(L, 1), T, (X,
  ↪ Y), _)}, time(ET), line(L).
18 arrive(train(L, N), SN, ET + TT) :- arrive(train(L, 1), SN, ET), shift(L, N, TT), N !=
  ↪ 1.
19 % define departure time as the latest timestep at which the train stays at the
  ↪ synchronized node

```

```

20 depart(train(L, 1), SN, LT) :- sync(SN, (X, Y)), LT = #max {T: at(train(L, 1), T, (X,
    ↪ Y), _)}, time(LT), line(L).
21 depart(train(L, N), SN, LT + TT) :- depart(train(L, 1), SN, LT), shift(L, N, TT), N !=
    ↪ 1.

```

Lines 15, 18, and 21 are atoms deduced from the constant shift in time of subsequent trains.

The constraints below ensure that the trains behave according to the requirements of TCP and the Flatland environment :

```

22 % train must dwell with a given duration at synchronized nodes
23 :- DT - AT != ST, dwell(L, ST), arrive(train(L, 1), SN, AT), depart(train(L, 1), SN,
    ↪ DT).
24 % train stops at each non-SN route cell for exactly one timestep
25 :- route(L, _, (X, Y)), not sync(_, (X, Y)), {at(train(L, 1), _, (X, Y), _)} != 1.
26 % train must end at the last cell on the route
27 :- ET = #max {T: at(train(L, 1), T, (_, _), _)}, LC = #max {C: route(L, C, (_, _))},
    ↪ route(L, LC, (X, Y)), not at(train(L, 1), ET, (X, Y), _).

```

Line 23 ensures that the train stays at synchronized nodes with a given dwell time. Assuming that the route does not form a loop, Line 25 provides that the train runs through the route without stopping. Line 27 states that the train must end at the last cell on the route.

As our main concern, the TCP objective can be expressed by the following rules:

```

28 % shortest waiting time constraint
29 sync1(train(L1, N1), train(L2, N2), SN) :- DT - (AT + ltransfer) >= swait,
    ↪ depart(train(L2, N2), SN, DT), arrive(train(L1, N1), SN, AT), L1 != L2.
30 % longest waiting time constraint
31 sync2(train(L1, N1), train(L2, N2), SN) :- DT - (AT + stransfer) <= lwait,
    ↪ depart(train(L2, N2), SN, DT), arrive(train(L1, N1), SN, AT), L1 != L2.
32 sync(T1, T2, SN) :- sync1(T1, T2, SN), sync2(T1, T2, SN).
33 % maximize synchronization
34 numSync(N) :- N = {sync(_, _, _)}.
35 #maximize {N: numSync(N)}.

```

Lines 29 and 31 are simply direct translations of equations 9 and 10 with the help of depart/3 and arrive/3 atoms. A synchronized connection is realized when these two constraints are fulfilled, and the total count of synchronized connections is maximized by the #maximize statement.

3.2 TCP with Path-finding

The encoding of TCP with path-finding is largely similar to that without path-finding, but a few differences respond to the expansion of the problem and the inclusion of non-synchronization stations. One of the differences is that the end terminal of each line is indicated explicitly with the end/2 atom provided by the instance, rather than being inferred by the #max statement from a set of at/4 atoms:

```

36 % train must end at the ending cell
37 :- ET = #max {T: at(train(L, 1), T, (_, _), _)}, at(train(L, 1), ET, (X, Y), _), not
    ↪ end(L, (X, Y)).

```

Apart from that, it is no longer necessary for the train to travel along a predetermined route, so the relevant rules are removed. Instead, trains must not stop at cells that are not stations, regardless of the final chosen route:

```
38 % train must not stop at cells that are not stations
39 :- not station(L, (X, Y)), at(train(L, 1), T, (X, Y), _), at(train(L, 1), T + 1, (X, Y),
    ↪ _).
```

Additionally, trains must pass through stations belonging to their lines, whether they are synchronized nodes or not:

```
40 % train must past through stations belonging to its line
41 :- station(L, (X, Y)), {at(train(L, 1), _, (X, Y), _)} = 0.
```

3.3 TCP with collision

To represent trains coming from different directions on the same line, we have introduced a direction attribute in train/2 atoms, as seen below. In this configuration, each cell is assumed to contain at least two parallel tracks to accommodate trains coming from opposite directions. In theory, it's also possible to create extra cells, but we have opted here to keep the representation simple.

```
42 % initialize position of the first train
43 at(train(L, 1, 1), T, (X, Y), D) :- first_start(L, 1, 1, T), start1(L, (X, Y), D).
44 at(train(L, 1, 2), T, (X, Y), D) :- first_start(L, 1, 2, T), start2(L, (X, Y), D).
```

The directions 1 and 2 stand for trains coming from opposite directions. Accordingly, these trains need different starting positions, as described in the following atoms, start1/3 and start2/3. The trains are required to move through the route predefined by the route/3 atoms, thus traveling through the line.

```
45 start1(1, (0, 2), s).
46 start2(1, (5, 2), n).
```

Lastly, an additional rule was added to prohibit trains from the same direction (on the same track) from being on the same position at the same timestep.

```
47 :- {at(train(_, _, _), T, (X, Y), TD)} > 1, at(train(_, _, _), T, (X, Y), TD).
```

4 Visualization

4.1 Notes on Flatland integration

At first, extending the official Flatland module to support creating train lines was considered, since this would have the added benefit of simplifying instance generation (i.e railway generation) as well as making use of the built-in visualizer. This turned out to be very difficult, since for TCP instances, the train atoms are only created in the solving process. Thus, trains would first be replaced by train lines in the instance, then added back in to the RailEnv environment after solving. Furthermore, synchronized nodes which are reachable from all train lines have to be found (i.e there exists at least one route passing through the start station, the synchronized node, and the end station). And lastly, the source code for the official visualizer is hard to understand and not easily extensible. All in all, we opted to create our instances manually and write our own visualizer using the Pillow library to bypass these problems.

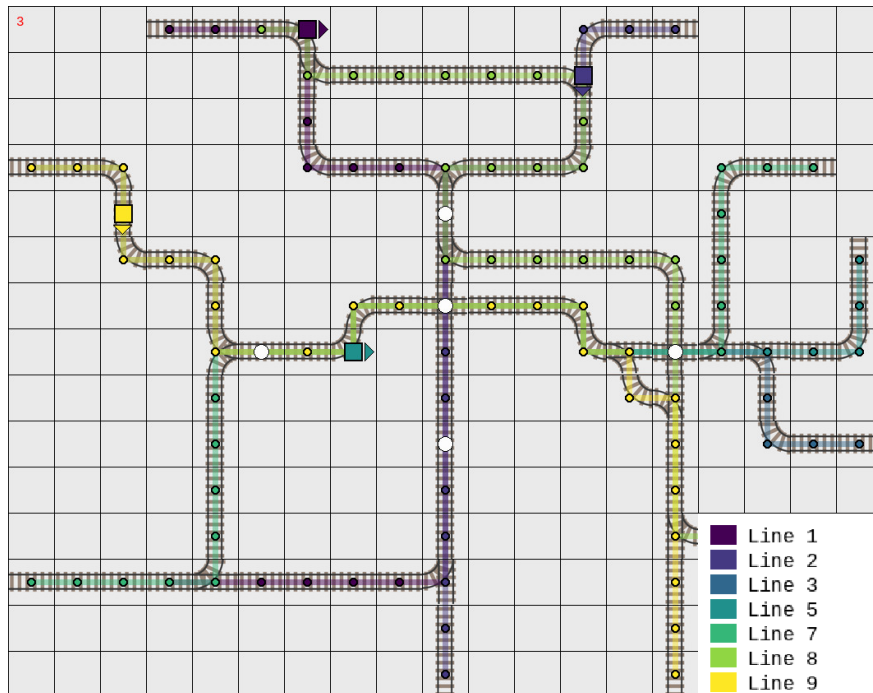


Figure 2: Example Frame

4.2 Implementation details

Generally, there are two approaches to implementing a visualizer for Flatland-related problems. We can either use the generated action atoms to track the location the trains or alternatively, use atoms which directly represent a train’s position. The first approach has the benefit of being compatible with other encodings for Flatland, since action atoms are standardized, but requires a more complex implementation. Additionally, this approach is not robust when the speed of trains are variable, e.g in case of acceleration. Even though these more advanced scenarios are not yet implemented in our project, the second approach was chosen in order to provide a more robust foundation for future groups who wish to tackle TCP. The source code for this can be found under the file ‘visualizer.py’ in our repository.

The central render function is as follows:

1. The track sprites are loaded and resized depending on the dimensions of the railway network
2. Cells are looped over and rendered with the sprites loaded previously
3. For each line, the route is extracted and the color is randomly generated, drawn as a transparent layer on top of the cells. Here, we aim to mimic the metro map lines.
4. If exists, synchronized nodes are drawn on top.
5. The previous layer is then kept as the background. We then looped over each timestep, and for each timestep, rendered each train’s position to create a single frame. An example can be seen in Figure 2.
6. Individual framed are then saved in an array, stitched together and exported as a gif.

5 Results & Future Works

In this project, we represent TCP in a modified Flatland environment and subsequently provide a valid ASP encoding according to the Flatland guidelines. Secondly, we have extended the Flatland environment to support TCP, as well as explored different variants of TCP, such as TCP with path-finding and TCP with collision, highlighting the potential of using ASP to solve TCP. However, our current ASP encoding exhibits limited performance and scalability, which remains to be improved upon. We have also considered several extensions to TCP, which were not iterated on due to the limited scope of this project. They are as follows:

- Different weights can be assigned to the entities of TCP to reflect the overall satisfaction of passengers. For example, larger rewards can be assigned to more important stations or lines due to a larger flow of travelers. Larger penalties could be imposed on departure-arrival time differences that are close to the boundaries of the interval set by equations (2) and (3) as opposed to those that are far from them. This addition allows a more detailed view of the problem with the use of weak constraints.
- To increase the resolution of the representation, bidirectional lines can be represented as two separate tracks rather than as a single track in the modified Flatland instance. This also results in stations that contain platforms occupying more than one cell in the grid. However, connection and collision have to be clearly differentiated under this fine-grained setup.
- Common extensions to Flatland can also be added to TCP, such as: Malfunctions, Speed and Acceleration.

Overall, we hope to provide a good groundwork for future groups seeking to tackle TCP.

A Appendix A

```
1 line(1..2).
2 headway(1, 8).
3 headway(2, 6).
4 dwell(1, 3).
5 dwell(2, 2).
6 cell((3, 2), 33825).
7 cell((0, 2), 32800).
8 cell((1, 2), 32800).
9 cell((2, 2), 32800).
10 cell((4, 2), 32800).
11 cell((5, 2), 32800).
12 cell((3, 0), 1025).
13 cell((3, 1), 1025).
14 cell((3, 3), 1025).
15 cell((3, 4), 1025).
16 start(1, (0, 2), s).
17 start(2, (3, 0), e).
18 sync(1, (3, 2)).
19 route(1, 0, (0, 2)).
20 route(1, 1, (1, 2)).
21 route(1, 2, (2, 2)).
22 route(1, 3, (3, 2)).
23 route(1, 4, (4, 2)).
24 route(1, 5, (5, 2)).
25 route(2, 0, (3, 0)).
26 route(2, 1, (3, 1)).
27 route(2, 2, (3, 2)).
28 route(2, 3, (3, 3)).
29 route(2, 4, (3, 4)).
30 period(20).
31 #const lwait = 10.
32 #const swait = 0.
33 #const ltransfer = 2.
34 #const stransfer = 1.
```

Listing 1: ASP code for simple instance

References

- [1] Yusak O Susilo and Oded Cats. “Exploring key determinants of travel satisfaction for multi-modal trips by different traveler groups”. In: *Transportation Research Part A: Policy and Practice* 67 (2014), pp. 366–380.
- [2] Tao Liu, Oded Cats, and Konstantinos Gkiotsalitis. “A review of public transport transfer coordination at the tactical planning phase”. In: *Transportation Research Part C: Emerging Technologies* 133 (2021), p. 103450.
- [3] Florian Laurent et al. “Flatland competition 2020: MAPF and MARL for efficient train coordination on a grid world”. In: *NeurIPS 2020 Competition and Demonstration Track*. PMLR. 2021, pp. 275–301.

- [4] Karthikgeyan Sivakumaran et al. “Cost-saving properties of schedule coordination in a simple trunk-and-feeder transit system”. In: *Transportation Research Part A: Policy and Practice* 46.1 (2012), pp. 131–139.
- [5] Zhichao Cao et al. “Optimal synchronization and coordination of actual passenger-rail timetables”. In: *Journal of intelligent transportation systems* 23.3 (2019), pp. 231–249.
- [6] Sharada Mohanty et al. *Flatland-RL : Multi-Agent Reinforcement Learning on Trains*. 2020. arXiv: 2012.05893 [cs.AI].